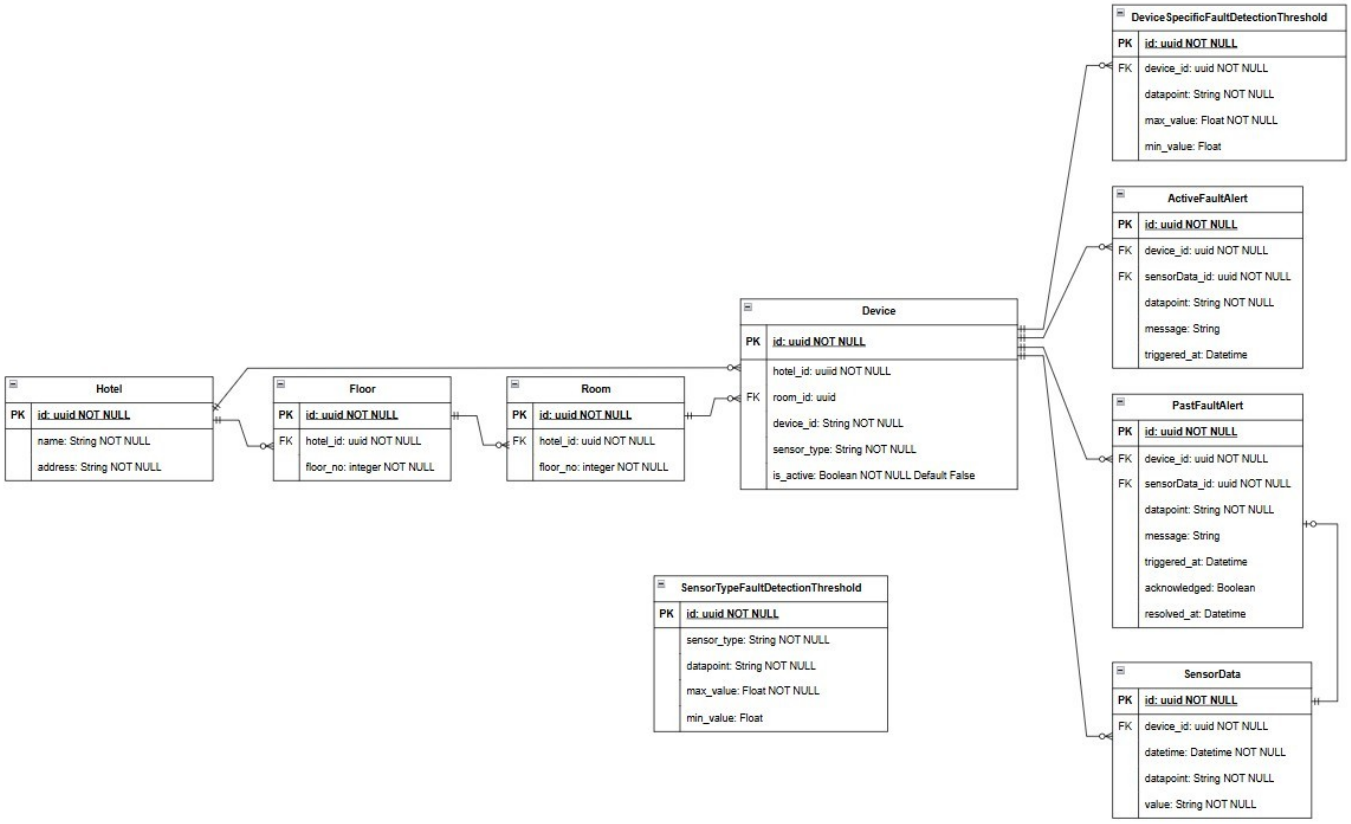
ER Diagram

This schema supports a fault detection system for Automatic Fault Detection and Diagnostic (AFDD). It allows

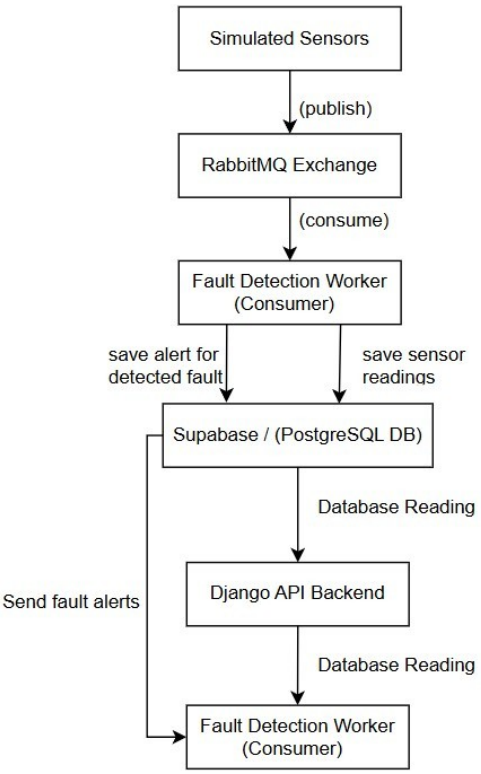
- Hierarchical representation of hotels, floors, and rooms
- Device installation and monitoring
- Storage of time-series sensor data
- Rule-based fault detection using thresholds (default and per-device)
  - SensorTypeFaultDetectionThreshold → default threshold values
  - DeviceSpecificFaultDetectionThreshold → threshold specific for each device (overriding default threshold values)
- Real-time and historical fault alert tracking



System Flow

1. Simulated Sensors

- Python agent **simulating sensor data** for devices deployed in hotel rooms or other locations.
- They **generate real-time readings** such as temperature, humidity, CO_2 levels, etc.
- The data is then **published** to a RabbitMQ exchange.



2. RabbitMQ Exchange

- Acts as the **message broker** in the system.
- It receives the incoming sensor data and **routes it to the correct queues.**

3. Fault Detection Worker

- Python agent that **subscribes to messages from RabbitMQ.**
- It processes incoming sensor data and **checks it against fault thresholds**, which may be:
  - Global thresholds (SensorTypeFaultDetectionThreshold)
  - Device-specific overrides (DeviceFaultDetectionThreshold)
  - If a **fault is detected**, it triggers further action to create the fault alert and save in PostgreSQL DB hosted by Supabase
- The incoming sensor data is always saved in database for historical fault occurrence checking

4. Supabase / PostgreSQL (Database Layer)

- The fault alerts are persisted in a PostgreSQL database (Supabase is used here as a hosted DB + backend platform).
- Alerts are stored in two tables:
  - **ActiveFaultAlert**: for currently active faults.
  - **PastFaultAlert:** for archived, acknowledged, or resolved faults.
- Uses **Supabase's Subscribe-to-Change functionality** to provide real-time alert notifications by listening to fault alert database change

5. Django Backend (API Layer)

- The Django server **exposes APIs** that interact with the database.
- Responsibilities include:
  - Serving fault alerts data (active/past)
  - Managing hotels, rooms, devices, and thresholds
  - Acknowledging or resolving alerts

7. React Frontend (UI Layer)

- The React-based frontend displays the data from the backend APIs.
- It may include features like:
  - Real-time fault alerts dashboard
  - Acknowledgement/resolution UI
  - Device monitoring per hotel/room
  - Threshold configuration interface

Implementation

Due to time constraints, not all planned features were fully implemented. However, the core components of the system were developed and integrated successfully to demonstrate real-time fault detection and alerting.

Backend: Django and Supabase

- **Supabase** serves as the PostgreSQL database, providing both storage and real-time change tracking capabilities.
- **Django** is used as the backend framework to expose REST APIs for fault data access and integration.
- Django is also used to define models for scheme
- **Implemented API Endpoints:**
  - GET /fault-detection/active-alarms/ – Retrieves a list of all current active alarms.
  - GET /fault-detection/active-fault-alarms/ – Retrieves a list of all alarms for which fault was detected.
- **Real-Time Fault Notifications:**
  - **Supabase's Subscribe-to-Change functionality** was used to provide real-time notice when the fault alert is created to the front-end

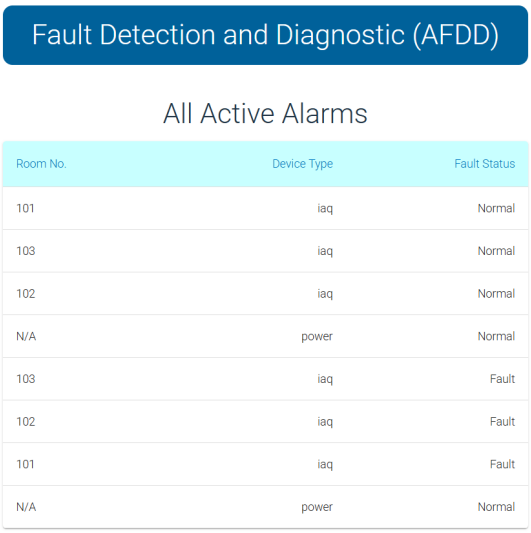
Python Agents

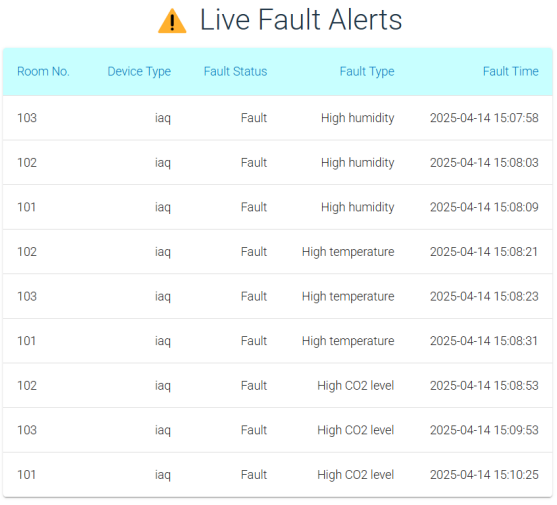
Two Python-based agents were developed to simulate and process sensor data:

1. sensor_simulator.py → stimulating sensor data
   - Simulates live sensor data (e.g., temperature, humidity, CO_2).
   - Publishes the data to a RabbitMQ exchange.
2. worker.py → subscribing to the data published
   - Subscribes to sensor data from RabbitMQ.
   - Applies fault detection logic using threshold values.
   - Stores sensor data and fault alerts directly into Supabase's PostgreSQL via Django ORM.

- Both scripts are integrated within the Django project in 'faultDetection' app under agent folder to reuse models and maintain consistency with the database schema.

Frontend: React Application





- The frontend is built using **React** to provide a modern and responsive UI for fault monitoring.
- **Implemented Features:**
  - **Active Alarms View** – Displays all current active faults in a structured format.
  - Live Fault Notifications – Real-time fault alerts are shown instantly using Supabase's change subscription mechanism.
- When a new fault is detected and inserted into the ActiveFaultAlert table, the frontend:
  - Updates the alarm list live.
  - Displays a visual notification (e.g., banner or toast) to inform the user.